default Predicate<T> and(Predicate<T> p) :
* It is a default method of Predicate<T> interface.
* It is mainly used to join two predicates and return Predicate<T>

```
package com.ravi.predicate;

import java.util.function.Predicate;

//Verify whether my name starts with 'R' character in mail id name length is 12

public class PredicateDemo
{
    public static void main(String[] args)
    {
        Predicate<String> startsWith = name -> name.startsWith("R");
        Predicate<String> nameLength = name -> name.length()==12;

        //join both the predicate

        Predicate<String> p1 = startsWith.and(nameLength);

        boolean isValid = p1.test("Ravi");

        SO.println(isValid);

    }

}
```

Consumer<T> functional interface :
It is a predefined functional interface available in java.util.function sub package.

It provides an abstract method accept() which takes T type parameter and returns nothing (void). It is used to accept the parameter value or consume the value.

```
@FunctionalInterface
public interface Consumer<T>
{
  void accept(T t);
}
```

Program:-

```
package com.ravi.consumer;

import java.util.function.Consumer;

public class ConsumerDemo {

    public static void main(String[] args)
    {
        Consumer<Employee> c1 = emp -> SO.println(emp);
        C1.accept(new Employee(111, "Ravi"));

        Consumer<Boolean> c2 = isValid -> SO.println(isValid);
        C2.accept(true);

        Consumer<Double> c3 = num -> SO.println(num);
        C3.accept(23.90);

    }

}

class Employee
{
    private int id;
    private String name;

    public Employee(int id, String name)
    {
        super();
        this.id = id;
        this.name = name;
    }

    @Override
    public String toString()
    {
        return "Employee [id= " + id + ", name= " + name + "]";
    }

}
```

Function<T,R> functional interface :
It is a predefined functional interface available in java.util.function sub package.

```
@FunctionalInterface
public interface Function<T,R>
{
  R apply(T t);
}
```

Type Parameters:
T - the type of the INPUT to the function
R - the type of the RESULT of the function.

It provides an abstract method apply that accepts one argument(T) and produces a result(R).

Note :- The type of T(input) and the type of R(Result) both will be decided by the developer.

//Program
package com.ravi.function;

```
import java.util.function.Function;

//Find the length of the given city

public class FunctionDemo1
{
    public static void main(String[] args)
    {
        Function<String,Integer> f1 = (ity -> city.length());

        String cityName = SC.nextLine("Enter your city name :");
        Integer cityLength = f1.apply(cityName);

        SO.println("Length of "+cityName+" is :"+cityLength);
    }
}
```

```
package com.ravi.function;

import java.util.function.Function;

//Verify whether my name is Ravi/ravi or not

public class FunctionDemo2
{
    public static void main(String[] args)
    {
        Function<String,Boolean> f1 = name -> name.equalsIgnoreCase("Ravi");
        String myName = SC.nextLine("Enter your name :");
        SO.println("Are you Ravi ?:"+f1.apply(myName));
    }

}
```

```
package com.ravi.function;

import java.util.function.Function;

class Employee

    private int id;
    private String name;
    private double salary;
    public Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public double getSalary() {
        return salary;
    }
    public void setSalary(double salary) {
        this.salary = salary;
    }
    @Override
    public String toString() {
        return "Employee [id= " + id + ", name= " + name + ", salary= " + salary + "]";
    }
}

// want to fetch Employee Salary

public class FunctionDemo3 {

    public static void main(String[] args)
    {
        Function<Employee,Double> f1 = emp -> emp.getSalary();
        Employee e = new Employee(111, "Ratn", 90000);

        Double salary = f1.apply(e);
        SO.println("Employee Salary is :"+salary);
    }

}
```

Supplier<T> Functional interface :
It is a predefined functional interface available in java.util.function sub package.

It provides an abstract method get() which does not take any argument but produces/supply/return a value of type T.

```
@FunctionalInterface
public interface Supplier<T>
{
  T get();
}
```

Program

```
package com.ravi.supplier_demo;

import java.util.function.Supplier;

public class SupplierDemo1
    public static void main(String[] args)
    {
        Supplier<String> s1 = () -> System.lineSeparator();
        SO.println(s1.get());
    }

package com.ravi.supplier_demo;

import java.util.function.Supplier;

class Player

    private int id;
    private String name;

    public Player(int id, String name)
    {
        super();
        this.id = id;
        this.name = name;
    }

    @Override
    public String toString()
    {
        return "Player [id= " + id + ", name= " + name + "]";
    }

}

public class SupplierDemo2
    public static void main(String[] args)
    {
        Supplier<Player> s1 = () ->
        {
            return new Player(101, "John");
        };

        Player player = s1.get();
        SO.println(player);
    }
}
```

```
package com.ravi.supplier_demo;

import java.util.function.Supplier;

class Customer

    private int id;
    private String name;
    private double bill;

    public Customer(int id, String name, double bill)
    {
        super();
        this.id = id;
        this.name = name;
        this.bill = bill;
    }

    @Override
    public String toString()
    {
        return "Customer [id= " + id + ", name= " + name + ", bill= " + bill + "]";
    }
}

public class SupplierDemo3 {

    public static void main(String[] args)
    {
        Supplier<Customer> s1 = () ->
        {
            int id = Integer.parseInt(SC.nextLine("Enter customer id :"));
            String name = SC.nextLine("Enter customer name :");
            double bill = Double.parseDouble(SC.nextLine("Enter customer Bill :"));

            Customer customer = new Customer(id, name, bill);
            return customer;
        };

        Customer customer = s1.get();
        SO.println(customer);

    }

}
```

TQ :-
WAP to generate 8 digits password by combining all the alphabets, digits and special character.

Random :
* It is a predefined class available in **java.util** package through which we can **generate random numbers** based on the **bound**.

public int nextInt(int bound)

* This method will generate a **random number** based on the given bound from 0 (inclusive) to bound (exclusive)

* We cannot pass 0 OR -ve number as a bound otherwise it will generate a runtime exception i.e. java.lang.IllegalArgumentException

```
package com.ravi.supplier_demo;

import java.util.Random;

public class RandomNumber {

    public static void main(String[] args)

        Random random = new Random();
        int randomNumber = random.nextInt(101);
        SO.println("randomNumber: " +" + " 0 - 100");
    }

}
```

Random Password Generation :

```
package com.ravi.supplier_demo;
import java.util.Random;
import java.util.function.Supplier;

public class SupplierDemo4 }

    public static void main(String[] args)

        String upper = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
        String lower = "abcdefghijklmnopqrstuvwxyz";
        String digit = "0123456789";
        String special = "!~@#$%^&*";  //!1

        String totalCharacter = upper + lower + digit + special;

        Supplier<String> pwdGen = () ->
        {
            Random random = new Random();
            String password = "";

            for(int i=1; i<=8; i++)
            {
                int randomNum = random.nextInt(totalCharacter.length());  //!1 10 to
70
                char ch = totalCharacter.charAt(randomNum);
                password += ch;
            }

            return password;
        };

        String password = pwdGen.get();
        SO.println(password);

    }

}
```

Can we create our own user-defined functional interfaces :-

Yes, We can create our own user-defined functional interfaces as shown below.

```
package com.ravi.user_defined_FI;

@FunctionalInterface
public MyFunctInterface1_0_0_R
{
  R myApply(T a, U b, V c);
}

public class CustomFunctionalInterface {

    public static void main(String[] args)
    {
        MyFunctInterface<Integer, Integer, Integer, String> myFun =
        (a,b,c) -> "a = " + a + c;

        SO.println(myFun.myApply(100, 200, 300));
    }

}
```